Practical – 02

Name: Anuj Dhanke

Roll no.: 20

Batch: A1

Title: Write a program to implement Huffman Encoding using a greedy strategy.

Input:

```
import heapq
from collections import Counter

# Node for Huffman Tree
class Node:
    def __init__(self, char, freq):
        self.char = char
        self.freq = freq
        self.left = None
        self.right = None

    def __lt__(self, other):
        return self.freq < other.freq


# Generate Huffman Codes
def generate_codes(root, code="", codes=None):
    if codes is None:
        codes = {}

    if root is None:
        return codes

    # Leaf node
    if root.char is not None:
        codes[root.char] = code
        return codes

    generate_codes(root.left, code + "0", codes)
```

```
    generate_codes(root.right, code + "1", codes)

    return codes


# Huffman Encoding
def huffman_encoding(text):

    # Count frequency of each character
    frequency = Counter(text)

    # Create a min heap
    heap = []

    for char, freq in frequency.items():
        heapq.heappush(heap, Node(char, freq))

    # Build Huffman Tree
    while len(heap) > 1:

        # Remove two nodes with lowest frequency
        left = heapq.heappop(heap)
        right = heapq.heappop(heap)

        # Combine them
        new_node = Node(None, left.freq + right.freq)
        new_node.left = left
        new_node.right = right

        # Add combined node back to heap
        heapq.heappush(heap, new_node)

    # Root of Huffman Tree
    root = heap[0]

    # Generate binary codes
    codes = generate_codes(root)

    # Encode the text
    encoded_text = ""
```

```
    for char in text:
        encoded_text += codes[char]

    return frequency, codes, encoded_text


# Main program
text = input("Enter text: ")

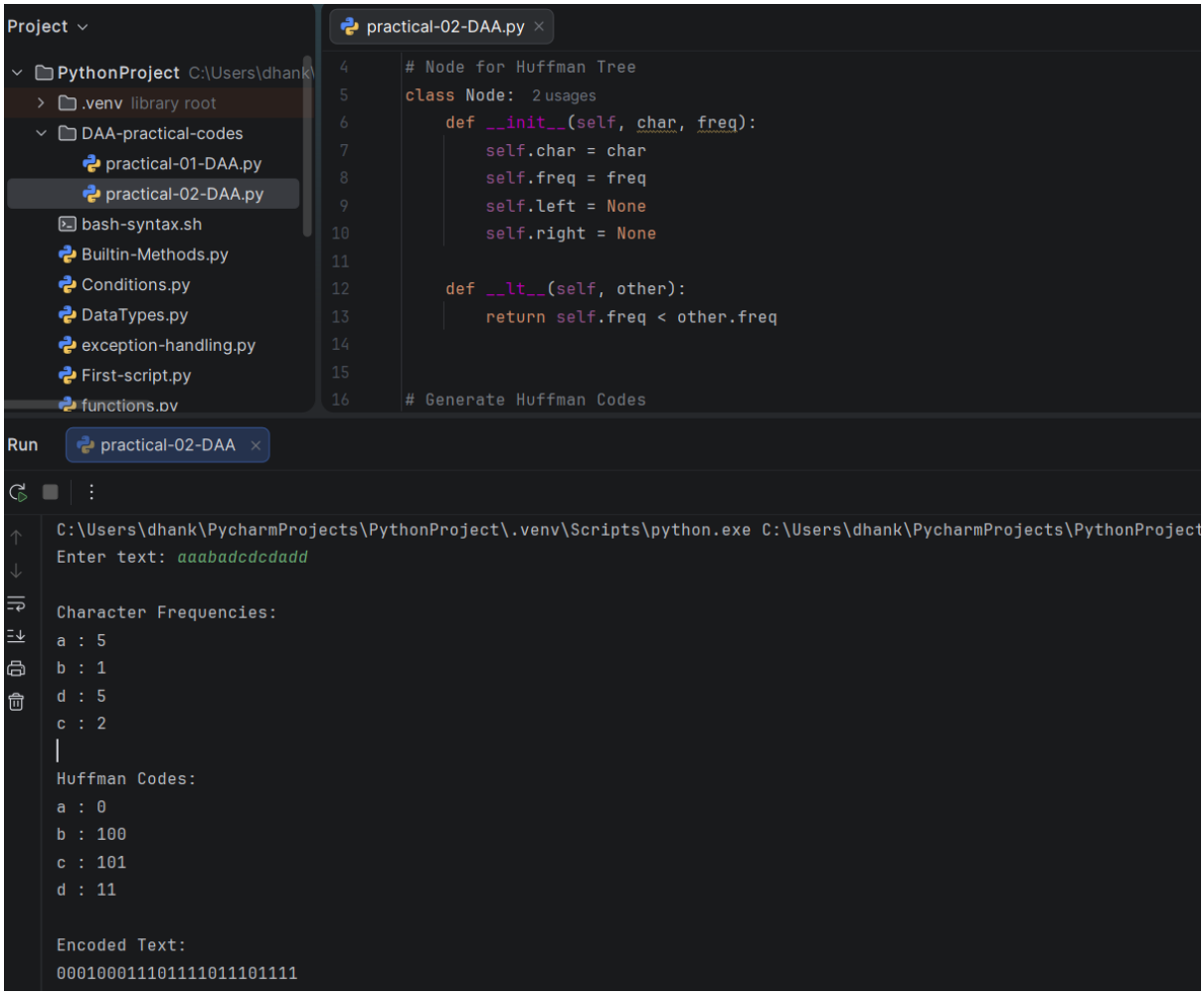
frequency, codes, encoded_text = huffman_encoding(text)

print("\nCharacter Frequencies:")
for char, freq in frequency.items():
    print(char, ":", freq)

print("\nHuffman Codes:")
for char, code in codes.items():
    print(char, ":", code)

print("\nEncoded Text:")
print(encoded_text)
```

Output:


Project
PythonProject C:\Users\dhank\
.venv library root
DAA-practical-codes
practical-01-DAA.py
practical-02-DAA.py
bash-syntax.sh
Builtin-Methods.py
Conditions.py
DataTypes.py
exception-handling.py
First-script.py
functions.py
practical-02-DAA.py
Node for Huffman Tree
class Node: 2 usages
def __init__(self, char, freq):
self.char = char
self.freq = freq
self.left = None
self.right = None
def __lt__(self, other):
return self.freq < other.freq
Generate Huffman Codes
Run
practical-02-DAA
C:\Users\dhank\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\dhank\PycharmProjects\PythonProject
Enter text: aaabadcdcdadd
Character Frequencies:
a : 5
b : 1
d : 5
c : 2
Huffman Codes:
a : 0
b : 100
c : 101
d : 11
Encoded Text:
000100011101111011101111